

ДИСЦИПЛИНА	Технологии разработки серверных приложений
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Текущий контроль
ПРЕПОДАВАТЕЛЬ	Дворецкий Артур Геннадьевич
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Ссылка на материал:

https://github.com/dvOretsky/fastapi-tutorial/blob/main/FAP1_KR1/FAP1_KR1.md

Контрольная работа №1 🖐️

🧠 Как сдавать работу

1. Выполните задания в отдельном репозитории (рекомендуется: один репозиторий на контрольную работу).
2. Загрузите исходный код в репозиторий (**commit** + **push**).
3. В **СДО** прикрепите отчёт в виде **ссылки на репозиторий с решением**.

Рекомендуется, чтобы репозиторий содержал понятную структуру и запускаемый проект (например, **app.py**, **models.py**, дополнительные файлы/папки при необходимости).

🌟 Задание 1.1

Создайте новый файл **app.py** и напишите код для **FastAPI-приложения**, которое при обращении к корневому URL (/) будет возвращать **JSON-ответ** **`{"message": "Добро пожаловать в моё приложение FastAPI!"}`**.

Дополнительно:

- Запустите приложение с помощью **uvicorn** в режиме разработки (с флагом автоперезагрузки).
- Откройте браузер и проверьте, корректно ли отображается ваш ответ.
- Без остановки **uvicorn**'а измените в коде текст ответа на **JSON**: **`{"message": "Авторелоад действительно работает"}`** и проверьте через браузер, что теперь возвращается новое сообщение.
- Измените имя переменной приложения (как во втором примере лекции) и убедитесь, что теперь **uvicorn** требует другой аргумент для запуска.

🌟 Задание 1.2

1. Создайте **html-файл** (напр. **`index.html`**), в тексте которого напишите:

```
<!DOCTYPE html>

<html lang="ru">
<head>

<meta charset="UTF-8">

<title> Пример простой страницы html</title>
</head>

<body>

Я ОБОЖАЮ ВСТАВЛЯТЬ К ПЕРВОЙ ПАРЕ :)
</body>

</html>
```

2. Создайте приложение **FastAPI**, которое принимает **GET-запрос** к дефолтной конечной точке (маршруту, адресу странички) `/` и возвращает html-страницу.
3. Сохраните файл и запустите приложение с помощью **uvicorn**:

```
uvicorn main:app --reload
```

4. Откройте `http://localhost:8000` в вашем веб-браузере.

Для выполнения задания можно прочитать про возможность **FastAPI** `"FileResponse"`.
Сделать это можно например [тут!](#)

🌟 Задание 1.3*

Создайте приложение **FastAPI**, которое принимает **POST-запрос** к конечной точке (маршруту, адресу странички) `/calculate` с двумя числами (`num1` и `num2`) в качестве входных данных. Приложение должно ответить суммой двух чисел.

Например, запрос на `/calculate` с `num1=5` и `num2=10` должен возвращать `{"result": 15}` в ответе.

Примечание: Вы можете использовать любой метод по вашему выбору для запуска приложения **FastAPI** для тестирования конечной точки

🌟 Задание 1.4

Ваша задача состоит в том, чтобы расширить существующее приложение **FastAPI**, добавив в него следующее:

1. Определите в файле `models.py` **Pydantic модель** с именем **"Пользователь"** (`"User"`) со следующими полями:
 - `name` (str);
 - `id` (int).
 2. Импортируйте модуль `models.py` в основной файл приложения и создайте в нем экземпляр класса `User`, с соответствующими полями модели `User`:
 - `name`: "Ваше Имя и Фамилия";
 - `id`: 1.
 3. Реализуйте **функцию**, которая при получении **GET-запроса** по дополнительному маршруту `/users` возвращала бы **JSON** с данными о пользователе (юзере).
-

🌟 Задание 1.5*

Ваша задача состоит в том, чтобы расширить существующее приложение **FastAPI**, добавив новую конечную точку **POST**, которая принимает данные **JSON**, представляющие пользователя, и возвращает те же данные с дополнительным полем, указывающим, является ли пользователь взрослым или нет.

1. Определите **Pydantic модель** с именем **"Пользователь"** (`"User"`) со следующими полями:
 - `name` (str);
 - `age` (int).
2. Создайте новый маршрут `/user`, который принимает запросы **POST** и принимает полезную нагрузку **JSON**, содержащую пользовательские данные, соответствующие модели `User`.
3. Реализуйте **функцию** для проверки того, является ли пользователь взрослым (возраст ≥ 18) или несовершеннолетним (возраст < 18).
4. Верните пользовательские данные вместе с **дополнительным полем** `is_adult` в ответе **JSON**, указывающим, является ли пользователь взрослым (`True`) или несовершеннолетним (`False`).

Пример: Запрос в формате **JSON**:

```
{
  "name": "Ваше Имя и Фамилия",
  "age": 25
}
```

Ответ в формате **JSON**:

```
{
  "name": "Artur Dvoretzky",
```

```
"age": 26,  
"is_adult": true  
}
```

★ Задание 2.1

Расширьте существующее приложение **FastAPI**, создав конечную точку **POST**, которая позволяет пользователям отправлять отзывы. Конечная точка должна принимать данные **JSON**, содержащие имя пользователя и сообщение обратной связи.

1. Определите **Pydantic модель** с именем **"Feedback"** (обратная связь) со следующими полями:

- **name** (str);
- **message** (str).

2. **Создайте новый маршрут** публикации **"/feedback"**, который принимает данные **JSON** в соответствии с моделью **Feedback**.

3. Реализуйте **функцию** для обработки входящих данных обратной связи и ответа сообщением об успешном завершении.

4. **Сохраните** данные обратной связи в списке или хранилище данных, чтобы отслеживать все полученные отзывы.

Пример: Запрос **JSON**:

```
{  
  "name": "Rustam",  
  "message": "Отличный день! Мне нравится ходить в школу!"  
}
```

Ответ **JSON**:

```
{  
  "message": "Feedback received. Thank you, Rustam."  
}
```

★ Задание 2.2*

Расширьте приложение **FastAPI**, добавив валидацию отзывов и запрет на использование нежелательных слов. Новая версия конечной точки **/feedback** должна принимать **JSON** с отзывом, проверять корректность и сохранять его.

1. Определите **Pydantic модель** с именем **Feedback** со следующими ограничениями:

- **name** — строка, обязательное поле, от 2 до 50 символов;
 - **message** — строка, обязательное поле, от 10 до 500 символов.
2. Добавьте **кастомную валидацию** поля **message**, которая будет отклонять отзывы, содержащие недопустимые слова: "кринж", "рофл", "вайб" (в любых падежах). Если слово найдено — выбрасывайте **ValueError** с сообщением **"Использование недопустимых слов"**.
3. Создайте **POST-маршрут /feedback**, который:
- Принимает **JSON** по модели **Feedback**;
 - При успехе сохраняет данные в список **feedbacks**;
 - Возвращает **JSON** с сообщением: **{"message": "Спасибо, {name}! Ваш отзыв сохранён."}**.

Пример запроса:

```
{
  "name": "Артур",
  "message": "Это тяжело, но я справлюсь!"
}
```

Пример ответа:

```
{
  "message": "Спасибо, Артур! Ваш отзыв сохранён."
}
```

Пример некорректного запроса:

```
{
  "name": "A",
  "message": "Какой-то кринж у вас тут происходит..."
}
```

Пример ответа (HTTP 422):

```
# структура может отличаться в зависимости от версии пидантика

{
  "detail": [
    {
      "type": "string_too_short",
      "loc": [
        "body",

```

```
        "name"
    ],
    "msg": "String should have at least 2 characters",
    "input": "A",
    "ctx": {
        "min_length": 2
    }
},
{
    "type": "value_error",
    "loc": [
        "body",
        "message"
    ],
    "msg": "Value error, Использование недопустимых слов",
    "input": "Какой-то кринж у вас тут происходит...",
    "ctx": {
        "error": {}
    }
}
]
```

Материалы для изучения

- [Документация Pydantic 2.0: валидаторы](#) и [документация: поля](#);
- [Валидация данных в FastAPI \(Habr\)](#).

Для успешного выполнения вам пригодятся знания о `min_length`, `max_length` и `@field_validator`, а также о [регулярных выражениях](#) в Python.

Made with ❤️ by **dvOretsky**